

FOUNDATIONS OF LARGE LANGUAGE MODELS

Detailed NPTEL Examination Study Guide

Transformers • Self-Attention • Decoder-Only LLMs • In-Context Learning • LoRA • RAG • MIPS • Agentic RAG • GraphRAG

Prepared from the two uploaded IISc lecture PDFs, expanded into full-sentence, exam-oriented notes. The lecture material is the primary source; explanatory passages are included to make the concepts understandable from first principles.

Lecture material	Coverage
Topics in Large Language Models — Lecture Series, Part I	Transformer foundations, self-attention, multi-head attention, Transformer layers, decoder-only models, LLMs, in-context learning, LoRA
LLMs — Retrieval Augmented Generation	Parametric memory, RAG, RAG mathematics, retrievers, BERT encoders, MIPS, Agentic AI/RAG, GraphRAG

Primary lecture attribution: Sriram Ganapathy, Electrical Engineering, IISc. The uploaded slides identify the first lecture as Part I and the second as an LLM/RAG lecture. filecite markers are not embedded in the PDF; slide/page references are given throughout.

Contents

1. How to Use This Guide for NPTEL
2. From Tokens to Transformer Representations
3. Self-Attention: Complete Mathematical Derivation
4. Multi-Head Attention
5. Transformer Processing Layer
6. Decoder-Only Language Models and Causal Attention
7. Large Language Models and Pretraining
8. Parametric Memory and Its Limitations
9. In-Context Learning
10. Low-Rank Adaptation (LoRA)
11. Retrieval-Augmented Generation (RAG)
12. RAG Probability Formulation
13. Retriever Design and MIPS
14. RAG vs Fine-Tuning vs ICL vs LoRA
15. Agentic AI and Agentic RAG
16. GraphRAG
17. Standard RAG vs Agentic RAG vs GraphRAG
18. High-Value Formula Sheet
19. NPTEL Exam Traps and Conceptual Distinctions
20. Practice Questions with Detailed Answers
21. Final Revision Checklist

Source note: The second uploaded PDF contains the lecture contents listing LLM recap, in-context learning, low-rank adaptation, and RAG; the first uploaded PDF concentrates on RAG fundamentals and recent RAG variants. See the uploaded lecture pages corresponding to these sections.

1. How to Use This Guide for NPTEL

NPTEL examinations often reward precise conceptual understanding rather than only the ability to repeat definitions. A question may present a matrix dimension, a short Transformer computation, a statement about causal masking, or a comparison between two adaptation methods. The safest preparation strategy is therefore to understand the mechanism first and memorize the compact mathematical form only after the mechanism is clear.

The lecture material moves through a deliberate conceptual progression. It first establishes the Transformer as the computational foundation of modern language models. It then explains decoder-only language modeling, which provides the basic mechanism behind many large language models. Once the model is understood, the lecture asks how a pretrained model can be adapted without completely retraining it, leading to in-context learning and low-rank adaptation. Finally, it asks how an LLM can use information that is not stored in its parameters, leading to retrieval-augmented generation and its more advanced forms.

For examination preparation, keep three levels separate. The first level is the definition: for example, MIPS means Maximum Inner Product Search. The second level is the mechanism: a query vector is compared with document vectors using an inner product and the highest-scoring documents are selected. The third level is the mathematical detail: $\text{score}(x,z) = q(x)^T d(z)$, followed by top-k selection. NPTEL can test any of these three levels.

The uploaded Part I lecture identifies the main Transformer building blocks as attention, residual connection, normalization, feed-forward layers, and positional encoding. It also emphasizes the big-picture idea that every input representation can attend to other representations.

`filecite[turn0file1][L36-L48]`

A useful mental rule for this entire chapter is: **attention** decides what information is relevant, the **feed-forward network** transforms representations, the **decoder objective** teaches next-token prediction, **retrieval** supplies external information, and **agents** decide when and how to use tools.

2. From Tokens to Transformer Representations

2.1 Tokens and vector representations

A language model cannot operate directly on words as symbolic strings. Text is first divided into tokens. A token may be a whole word, a subword, punctuation, or another unit chosen by the tokenization system. Each token is then represented by a numerical vector called an embedding. If a sequence contains T tokens and every token has D features, the sequence can be arranged as a matrix X of dimension $T \times D$.

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_T \end{bmatrix} \in \mathbb{R}^{(T \times D)}$$

The second lecture illustrates this idea with a sentence such as “I am a student of machine learning,” assigning a vector representation x_1 , x_2 , and so on to the individual tokens. The important point for an exam is that the rows of X correspond to positions/tokens, while the columns correspond to embedding features. `filecite[turn0file1][L51-L67]`

2.2 Why a Transformer needs more than embeddings

An embedding gives a token a useful numerical representation, but the same token can have different meanings depending on its context. The word “bank,” for example, can refer to a financial institution or the side of a river. A Transformer therefore needs a mechanism through which the representation of one token can incorporate information from other tokens. Self-attention provides this mechanism.

The lecture describes the Transformer at a high level as a system in which every input representation can attend to every other input representation, including itself. The output is a weighted combination of representations, with weights determined by the similarity between representations.

□filecite□turn0file1□L36-L48□

2.3 Positional information

Self-attention by itself does not inherently impose a sequential order on a collection of token vectors. The Transformer therefore needs positional information so that the model can distinguish between different token positions. The uploaded lecture's LLM recap diagram explicitly shows token embeddings combined with positional encoding before the representations enter the decoder blocks.

□filecite□turn0file0□L10-L30□

NPTEL EXAM NOTE: A common conceptual trap is to say that token embeddings alone tell the Transformer where a token occurs in the sentence. The lecture architecture explicitly includes positional encoding in addition to token embeddings.

3. Self-Attention: Complete Mathematical Derivation

3.1 Why Query, Key and Value projections are introduced

The lecture explains that directly using the input representations does not provide the desired learnable transformations. Instead, three transformed versions of the input are constructed: queries, keys, and values. These are obtained through learned linear projection matrices.

$$\begin{aligned}Q &= X W_Q \\K &= X W_K \\V &= X W_V\end{aligned}$$

If X has dimension $T \times D$ and the projected query, key, and value dimension is d , then W_Q , W_K , and W_V have compatible dimensions $D \times d$. Consequently, Q , K , and V each have dimension $T \times d$. This dimensional reasoning is one of the easiest ways to answer NPTEL questions that ask for the size of an intermediate matrix. The lecture explicitly gives $Q = XW_Q$, $K = XW_K$, and $V = XW_V$.

□filecite□turn0file1□L68-L85□

3.2 Intuition for Query, Key and Value

The query can be understood as a representation of what a token is looking for. The key can be understood as a representation of what another token offers for matching purposes. The value contains the information that will ultimately be aggregated. This terminology is an intuition rather than a separate semantic meaning imposed by the mathematics, but it is extremely useful for understanding the computation.

For a particular token t , its query q_t is compared with every key $k_1, k_2, \dots, k_T$. If the query and a particular key are strongly aligned in the learned representation space, their dot product is relatively large and that token receives a larger attention weight after softmax.

3.3 Constructing the attention-score matrix

Once Q and K have been formed, the model computes their matrix product QK^T . The result contains one score for every query-key pair. Because there are T queries and T keys, the result has dimension $T \times T$.

$$\begin{array}{ll} Q & : T \times d \\ K & : T \times d \\ K^T & : d \times T \end{array}$$

$$Q K^T : T \times T$$

The element in row i and column j is $q_i \cdot k_j$. Therefore, row i describes how strongly token i attends to each token j before normalization. The lecture demonstrates this explicitly for a query q_4 , showing dot products such as $q_4 \cdot k_1 = 14$, $q_4 \cdot k_2 = 3$, $q_4 \cdot k_3 = -2$, $q_4 \cdot k_4 = 15$, and so on.

[filecite\turn0file1\L102-L116](#)

3.4 Scaling the scores

The raw dot products are divided by the square root of the key dimension. The resulting operation is called scaled dot-product attention. The scaling prevents dot products from becoming excessively large as the vector dimension increases. Without suitable scaling, softmax can become extremely peaked, which is undesirable for stable optimization.

$$\text{Scaled scores} = Q K^T / \text{sqrt}(d_k)$$

The standard form is therefore:

$$\begin{aligned} \text{Attention}(Q, K, V) \\ = \text{softmax}(Q K^T / \text{sqrt}(d_k)) V \end{aligned}$$

The lecture presents the scaled dot-product operation using the query/key dimension in the denominator. [filecite\turn0file1\L76-L85](#)

NPTEL EXAM NOTE: Do not replace $\text{sqrt}(d_k)$ with d_k in the standard scaled dot-product attention equation. Also distinguish d_k , the key/query dimension per head, from D , the original embedding dimension.

3.5 Softmax converts scores into attention weights

The softmax function converts a row of attention scores into normalized positive weights. For a score z_i , softmax assigns $\exp(z_i)$ divided by the sum of exponentials over the relevant positions. The resulting weights sum to one. This allows the output representation to be interpreted as a weighted combination of value vectors.

$$\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$$

In the lecture's numerical illustration, the q_4 scores are transformed into weights approximately equal to 0.23, 0.00, 0.00, 0.64, 0.00, 0.09, and 0.03. The largest weight is assigned to the fourth token because its corresponding score is the largest among the illustrated values.

[filecite\turn0file1\L102-L136](#)

3.6 Weighted aggregation of values

The final step is multiplication of the normalized attention-weight matrix by V . For the fourth query, the output can be understood as a weighted sum of v_1 through v_T . A token therefore does not simply copy another representation; it constructs a new representation by combining information from the positions to which it assigns attention.

$$e_4 = a_{41} v_1 + a_{42} v_2 + \dots + a_{4T} v_T$$

This is the key conceptual meaning of self-attention: each output representation contains information gathered from the entire accessible context, with the contribution of each token controlled by learned similarity between queries and keys.

3.7 Full dimensional analysis

$$\begin{array}{ll} X & : T \times D \\ WQ & : D \times d \\ Q & : T \times d \end{array}$$

$$\begin{array}{ll} WK & : D \times d \\ K & : T \times d \end{array}$$

$$\begin{array}{ll} WV & : D \times d \\ V & : T \times d \end{array}$$

$$QK^T : T \times T$$

$$\text{softmax}(QK^T / \sqrt{d_k}) : T \times T$$

$$(T \times T)(T \times d) = T \times d$$

Therefore, the attention operation transforms a $T \times D$ input into an attention output whose internal per-head dimension is $T \times d$. In a multi-head architecture, these per-head outputs are concatenated and subsequently projected as part of the Transformer implementation.

NPTEL EXAM NOTE: If $T = 100$ and $d = 64$, Q and K are 100×64 , but QK^T is 100×100 . The attention-score matrix is quadratic in the sequence length.

4. Multi-Head Attention

4.1 Why one attention operation is not enough

Language contains several types of relationships simultaneously. A token may need to attend to another token because of grammatical structure, semantic similarity, an entity relationship, or a long-range dependency. Multi-head attention addresses this by performing attention in several learned representation subspaces.

The lecture illustrates multiple heads using different attention relationships. One head can focus on a relationship resembling “who,” another can capture “did what,” and another can capture “to whom.” These labels are intuitive illustrations rather than hard-coded functions: the actual roles of the heads are learned from data. [filecite turn0file1 L138-L148](#)

4.2 Mathematical form

For head k , separate query, key, and value projections are used. The attention output of the head is calculated using the same scaled dot-product formula.

$$E_k = \text{softmax}(Q_k K_k^T / \sqrt{d_k}) V_k$$

After the individual head outputs are computed, they are concatenated:

$$E = [E_1 \ E_2 \ \dots \ E_h]$$

The lecture notes the usual relationship $d = h \times d_k$, where d is the overall model dimension, h is the number of heads, and d_k is the dimension assigned to each head. [\[filecite\]turn0file1\[L149-L159\]](#)

4.3 Numerical dimension example

Suppose the model dimension is 512 and the Transformer uses 8 heads. If the heads divide the dimension evenly, each head has dimension $512/8 = 64$. Each head therefore performs attention in a 64-dimensional space, and the eight outputs are concatenated to recover a 512-dimensional representation.

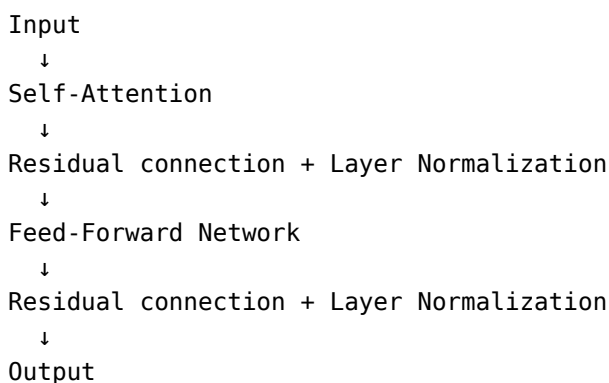
$$\begin{aligned}d &= 512 \\h &= 8 \\d_k &= d/h = 64\end{aligned}$$

NPTEL EXAM NOTE: The number of heads does not mean that the sequence length is divided among heads. Heads operate on different learned representation subspaces. The tokens remain present in every head.

5. Transformer Processing Layer

5.1 Overall structure

The Transformer block shown in the lecture consists of self-attention followed by an Add-and-Normalize operation, then a feed-forward network followed by another Add-and-Normalize operation. The diagram also shows residual paths around the sublayers. [\[filecite\]turn0file1\[L163-L175\]](#)



5.2 Residual connections

A residual connection allows a block to preserve a direct path from its input to its output. In a simplified form, if a sublayer computes $F(x)$, the residual result is $x + F(x)$. This structure helps information and gradients flow through deep networks. The lecture explicitly identifies improved gradient flow and stability as benefits of residual connections. [\[filecite\]turn0file1\[L168-L173\]](#)

5.3 Layer normalization

Layer normalization rescales and recenters activations across the feature dimensions of a representation. In a simplified mathematical form, a normalized vector is obtained by subtracting its mean and dividing by a standard-deviation term, followed by learned scale and bias parameters. The lecture describes LayerNorm in exactly this feature-wise normalization sense.

[\[filecite\]turn0file1\[L168-L170\]](#)

5.4 Feed-forward network

The feed-forward network provides a position-wise nonlinear transformation. Attention is responsible for mixing information across token positions, whereas the feed-forward sublayer transforms each position's representation after the cross-token information has been incorporated. A generic two-layer form is:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

The lecture's central architectural point is that self-attention and feed-forward processing are repeated across Transformer blocks. In the decoder-only LLM recap, each decoder block applies masked multi-head self-attention and a position-wise feed-forward network. [filecite\turn0file0\L19-L30](#)

6. Decoder-Only Language Models and Causal Attention

6.1 What makes a decoder-only model different?

The lecture describes decoder-only models as containing self-attention layers and emphasizes that their training objective is next-token prediction. Modern decoder-only LLMs use causal attention in an autoregressive setting. [filecite\turn0file1\L177-L205](#)

The word causal means that a token is prevented from using information from future positions when predicting the next token. During generation, the model should be able to use the prompt and everything it has already generated, but it must not see the future answer tokens because those tokens do not exist yet.

6.2 Autoregressive factorization

If a sequence contains x_1 through x_T , the probability of the entire sequence can be factorized as the product of conditional next-token probabilities. This is the mathematical foundation of autoregressive language modeling.

$$\begin{aligned} P(x_1, x_2, \dots, x_T) \\ = \prod_t P(x_t \mid x_1, \dots, x_{(t-1)}) \end{aligned}$$

At each position, the model learns to predict the next token from the preceding context. During training, the correct previous tokens are available, while during generation the model feeds its own generated tokens back into the next step.

6.3 Causal masking

A causal mask makes future positions inaccessible. For a four-token sequence, the conceptual mask can be represented as a lower-triangular matrix.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

The first position can attend only to itself. The second can attend to positions one and two. The fourth can attend to all four positions. In implementation, disallowed locations are assigned a very negative score before softmax so that their resulting attention weights become effectively zero.

NPTEL EXAM NOTE: Causal masking is not optional for autoregressive next-token prediction. Without it, the model could access future target tokens during training and the learning objective would be

compromised.

6.4 Language modeling as slot filling

The lecture makes an important conceptual observation: many language and creative generation tasks can be represented as structured next-token prediction or slot filling. For example, a question-answering prompt is transformed into a sequence in which the model generates the missing answer tokens. The same next-token mechanism can therefore support many apparently different tasks. [filecite turn0file1 L189-L205]

7. Large Language Models and Pretraining

The lecture describes LLMs as an extension of self-supervised learning to very large collections of data and models with millions to billions of parameters. Data can include text and, in broader multimodal systems, other modalities. The model is pretrained using objectives such as masked language modeling or next-token prediction, and the resulting parameters can later be adapted to supervised tasks. [filecite turn0file1 L222-L232]

7.1 What self-supervised learning means here

Self-supervised learning creates a learning signal from the data itself. In next-token prediction, the text already contains the target token. The model is given the preceding context and trained to predict what comes next. No human needs to manually label every individual token with a category such as “next word.”

7.2 Pretraining and adaptation

Pretraining gives the model broad capabilities. Adaptation then changes how the model behaves for a particular task or domain. The lecture introduces two important adaptation strategies: in-context learning, where the parameters remain frozen, and low-rank adaptation, where only a small number of additional parameters are learned. [filecite turn0file1 L271-L280]

8. Parametric Memory and Its Limitations

The RAG lecture describes knowledge stored inside model weights as parametric memory. It is knowledge learned during training and represented implicitly in the parameters. Once training is finished, this knowledge is not straightforwardly editable without additional training or fine-tuning. [filecite turn0file0 L47-L68]

8.1 Why parametric memory is useful

Parametric memory allows an LLM to answer questions without performing an explicit database lookup for every fact. The model has learned statistical and semantic patterns during training and can generate an answer directly from its internal representations.

8.2 Why parametric memory is insufficient

The lecture identifies four limitations. First, a model cannot automatically know constant changes in the external world after its training data was collected. Second, it may not be able to locate the source of a statement. Third, it can hallucinate, meaning it can generate plausible-sounding but unsupported information. Fourth, its capacity is finite. [filecite turn0file0 L85-L100]

These limitations motivate the use of non-parametric memory. Instead of forcing every piece of information to be permanently encoded in the model parameters, an external knowledge source can be updated independently. Retrieval can then bring relevant information into the model's context at inference time.

9. In-Context Learning

9.1 Definition

In-context learning is the ability of a pretrained model to perform a task based on examples included in the input context, without updating the model's parameters during that interaction. The lecture explicitly presents decoder-only models as capable of learning a new task purely from examples and emphasizes that the weights remain frozen. [\[filecite\]turn0file1\[L287-L317\]](#)

For example, a prompt can contain the pairs “Federer → Tennis” and “Kohli → ?”. The model can infer that the desired relationship is the sport associated with the person and answer “Cricket.” If the examples instead establish a country relationship, such as “Federer → Switzerland,” the same model can infer a different mapping for “Ronaldo → ?” and answer “Portugal.” [\[filecite\]turn0file1\[L287-L297\]](#)

9.2 Zero-shot, one-shot and few-shot

Type	Meaning	Example
Zero-shot	The model receives an instruction but no worked example.	Translate to French: apple → ?
One-shot	The model receives one input-output example and then a new input.	apple → pomme; banana → ?
Few-shot	The model receives several examples from which it infers the pattern.	cat → chat; dog → chien; house → maison; tree → ?

These definitions and examples are given in the lecture slides. [\[filecite\]turn0file1\[L329-L353\]](#)

9.3 How can ICL work if weights do not change?

The lecture explains that each Transformer layer can build an internal representation of the examples present in the context. Self-attention allows the model to dynamically retrieve relevant examples when processing a new query token. Thus, the model can display behavior resembling few-shot learning without performing a conventional parameter update. [\[filecite\]turn0file1\[L300-L317\]](#)

A useful way to visualize this is to imagine the prompt as temporary working memory. The pretrained parameters define the computation the model knows how to perform, while the examples in the current context provide temporary information that influences that computation. The information does not become a permanent change to the model's weights.

9.4 A deeper perspective: learning inside the forward pass

The lecture references work on regression-based in-context learning, where variables needed for a regression calculation can be stored in hidden activations during the forward pass. The important conceptual message is that a Transformer can implement computations that resemble a learning algorithm using its activations, even though the permanent model parameters remain fixed. [\[filecite\]turn0file1\[L355-L367\]](#)

NPTEL EXAM NOTE: Do not describe ICL as fine-tuning. ICL changes the information available to the current forward pass; fine-tuning changes learned parameters.

10. Low-Rank Adaptation (LoRA)

10.1 Motivation

Full fine-tuning of a large Transformer can require updating a very large number of parameters. The lecture introduces low-rank adaptation as a parameter-efficient alternative. The central idea is to freeze the pretrained matrix W_0 and learn a much smaller low-rank update. [\[filecite\]turn0file1\[L368-L386\]](#)

10.2 The LoRA equation

Suppose the original pretrained matrix is W_0 with dimensions $D \times d$. LoRA represents the change to that matrix as the product of two smaller matrices A and B . The resulting adapted matrix is:

$$\begin{aligned}\Delta W &= B A \\ W &= W_0 + \alpha \Delta W \\ &= W_0 + \alpha B A\end{aligned}$$

The lecture specifies A as $r \times d$ and B as $D \times r$, where r is much smaller than the original dimensions. Only A and B are learned from the data, while W_0 is frozen. α is a scaling factor. [\[filecite\]turn0file1\[L373-L386\]](#)

10.3 Why the parameter count becomes small

Full fine-tuning requires learning Dd parameters. LoRA requires learning the entries of A and B , which gives $r d + D r = r(D+d)$ trainable parameters. When r is small, this is dramatically smaller than Dd .

Full fine-tuning:
 $N_{\text{full}} = Dd$

LoRA:
 $N_{\text{LoRA}} = r(D + d)$

Parameter fraction updated:
 $r(D + d) / (Dd)$

10.4 Numerical example from the lecture

The lecture uses $D = 1024$, $d = 128$, and $r = 8$. Full fine-tuning would update $1024 \times 128 = 131072$ parameters. LoRA updates $8(1024 + 128) = 9216$ parameters. The ratio is approximately 0.0703, meaning about 7% of the parameters of that matrix are trainable under this simplified calculation. [\[filecite\]turn0file1\[L389-L400\]](#)

10.5 Where LoRA is typically inserted

The lecture states that LoRA is typically inserted into linear projection layers of Transformer blocks, especially the query and value projection matrices in self-attention. It can also be used in feed-forward layers, but the lecture describes this as less common. [\[filecite\]turn0file1\[L404-L415\]](#)

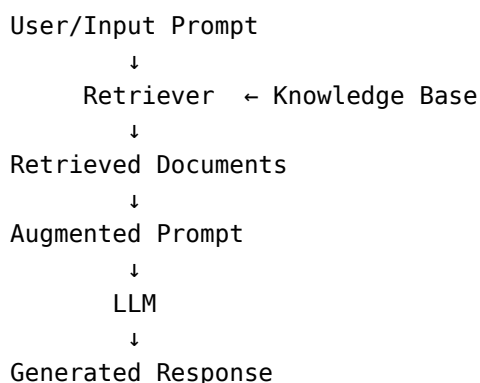
10.6 Why low-rank updates can be sufficient

A pretrained model already contains broad language representations. A downstream task often does not require replacing all of those representations. Instead, it may require a comparatively small shift that makes the existing model better suited to a new domain or task. LoRA constrains the learned update to a low-rank structure, making the adaptation much cheaper while preserving the frozen base model.

11. Retrieval-Augmented Generation (RAG)

11.1 Why RAG is needed

RAG is introduced as a way to extend the model with non-parametric memory. The external knowledge source can be updated without retraining the entire language model. The lecture's RAG architecture contains five conceptual stages: input prompt, retriever, retrieved documents, augmented prompt, and LLM-generated response. [\[filecite\turn0file0\L102-L116\]](#)



11.2 Retrieval

The retriever receives a query derived from the input and searches an external knowledge base. It selects a small set of documents or passages that are judged relevant to the query. The lecture uses the term top-k to describe this selection.

11.3 Augmentation

The retrieved snippets are appended to the original input. This creates an augmented prompt containing both the user's question and relevant external context. The lecture explicitly describes these retrieved snippets as acting as non-parametric memory. [\[filecite\turn0file0\L143-L160\]](#)

11.4 Generation

The LLM then receives the augmented input and generates a response conditioned on the retrieved context. In the lecture's Bangalore weather example, the retrieval stage supplies information that can be used to create a response about the current weather context rather than relying only on the model's internal knowledge. [\[filecite\turn0file0\L165-L182\]](#)

11.5 What RAG does not mean

RAG does not mean that the external documents are permanently written into the LLM's weights. It also does not mean that retrieval automatically makes the final answer correct. Retrieval quality, document quality, context selection, and generation quality all matter. A poor retriever can supply irrelevant evidence, and a generator can still misunderstand or misuse relevant evidence.

12. RAG Probability Formulation

The lecture gives a probabilistic formulation of RAG. Let x denote the input, y denote the output, d denote an individual document, and C denote the corpus. The probability of generating y given x can be expressed by marginalizing over possible documents.

$$P(y \mid x) = \sum_{(d \in C)} P(y \mid x, d) P(d \mid x)$$

The term $P(d \mid x)$ represents the retriever's relevance distribution: given the query x , how likely or relevant is document d ? The term $P(y \mid x, d)$ represents the generator's response distribution conditioned on both the original query and the retrieved document.

The exact sum over an entire corpus can be computationally impractical. Therefore, the lecture states that the conditioning is approximated using the top- k retrieved documents.

$$\begin{aligned} P(y \mid x) \\ \approx \sum_{(i=1 \text{ to } k)} P(y \mid x, d_i) P(d_i \mid x) \end{aligned}$$

This equation is valuable because it makes the division of responsibility between retrieval and generation explicit. The retriever decides which evidence is considered, while the generator determines how the answer is produced from that evidence. [filecite\[turn0file0L184-L202\]](#)

NPTEL EXAM NOTE: If a question asks what $P(d|x)$ represents, think retriever. If it asks what $P(y|x,d)$ represents, think generator conditioned on retrieved evidence.

13. Retriever Design and MIPS

13.1 Why encode queries and documents?

Searching a large collection by comparing raw text character by character is not enough for semantic retrieval. A retriever can instead map both the query and documents into a common vector space. Similar or relevant content should produce representations that receive high retrieval scores.

The lecture presents a simple BERT encoder for this purpose. The query is passed through a Transformer encoder, and the token representations are combined, in the illustrated approach by averaging, to obtain a representation for the entire query. Document passages can be encoded similarly. [filecite\[turn0file0L216-L234\]](#)

$$\begin{aligned} q(x) &= \text{Enc}_q(x) \\ d(z) &= \text{Enc}_d(z) \end{aligned}$$

13.2 Maximum Inner Product Search

Once query and document embeddings are available, the lecture uses the inner product as the relevance score. For query x and document z , the score is $q(x)^T d(z)$. The retriever then selects the documents with the largest scores.

$$\text{score}(x, z) = q(x)^T d(z)$$

$$Z_k(x) = \text{top-}k \text{ over } z \in C \text{ of } q(x)^T d(z)$$

The lecture identifies this retrieval operation as MIPS, or Maximum Inner Product Search. [filecite\[turn0file0L236-L254\]](#)

13.3 Small numerical example

Suppose the query embedding is $q = [2, 3]$ and one document embedding is $d1 = [4, 1]$. The inner product is $2 \times 4 + 3 \times 1 = 11$. If another document has $d2 = [1, 1]$, its score is $2 \times 1 + 3 \times 1 = 5$. Under this scoring rule, $d1$ ranks above $d2$.

$$\begin{aligned} q &= [2, 3] \\ d1 &= [4, 1] \\ q^T d1 &= 2 \cdot 4 + 3 \cdot 1 = 11 \end{aligned}$$

$$\begin{aligned} d2 &= [1, 1] \\ q^T d2 &= 2 \cdot 1 + 3 \cdot 1 = 5 \end{aligned}$$

NPTEL EXAM NOTE: The lecture specifically uses inner-product scoring. Do not automatically substitute Euclidean distance or cosine similarity when answering a question about the presented MIPS formulation.

14. RAG vs Fine-Tuning vs ICL vs LoRA

Method	Are base weights updated?	Where is task information stored?	Typical purpose
In-context learning	No	Current context / activations	Temporarily demonstrate a task or mapping through examples
LoRA	Base frozen; small matrices learned	Low-rank parameter update	Efficient task/domain adaptation
Full fine-tuning	Yes	Updated model parameters	Strong task/domain adaptation when training resources permit
RAG	Usually no	External knowledge base + current context	Ground answers in external/current/domain-specific information

The lecture directly contrasts frozen-parameter in-context learning with adaptation using a small number of parameters. It then introduces RAG as a mechanism for non-parametric memory.

[filecite\[turn0file1\[L271-L280\]](#) [filecite\[turn0file0\[L85-L116\]](#)

14.1 A simple decision framework

If the problem is primarily about showing the model a pattern for one interaction, in-context learning is the natural concept. If the model needs persistent task-specific adaptation but updating all parameters is too expensive, LoRA is appropriate. If the problem is that the model needs information that lives outside its weights and can change independently, RAG addresses that need. Full fine-tuning is the broadest parameter-update strategy among these choices.

NPTEL EXAM NOTE: A question may deliberately mix the terms “adaptation” and “knowledge.” Remember that LoRA is a parameter-efficient adaptation method, while RAG is fundamentally an external retrieval mechanism.

15. Agentic AI and Agentic RAG

15.1 What is an agent?

The lecture defines an agentic system as an AI that pursues a goal through a loop of reasoning and action rather than answering one prompt and stopping. The diagram shows a goal entering an agent, followed by an action such as calling a tool, an observation of the result, and either another iteration or a final output. [\[filecite\turn0file0\L278-L303\]](#)

```
Goal
↓
Agent / LLM controller
↓
Action: call a tool
↓
Observation: inspect result
↓
If goal not met → loop
If goal met → final output
```

The essential feature is not simply that an LLM can call a tool. The stronger idea is that the system can decide what to do next based on what it has learned from the previous action. This makes the workflow iterative.

15.2 Tool selection

The lecture illustrates an agent choosing among a vector store, web search, and SQL/API tools. A single user question may contain several sub-questions, and different tools may be appropriate for different sub-questions. The agent can call more than one tool before producing the final response.

[\[filecite\turn0file0\L256-L276\]](#)

15.3 Agentic RAG

Agentic RAG extends standard retrieval by allowing one or more retrieval cycles. The agent may reformulate the original query, decompose it into sub-questions, choose among multiple information sources, evaluate what it found, and retrieve again if the evidence is insufficient.

This makes agentic RAG particularly useful for multi-hop, ambiguous, or open-ended questions. The cost is that every additional model call, retrieval operation, or external tool invocation can increase latency and computational expense. The lecture explicitly contrasts the lower cost of one-pass standard RAG with the higher cost of multiple calls in agentic RAG. [\[filecite\turn0file0\L308-L335\]](#)

15.4 Standard RAG versus Agentic RAG

Dimension	Standard RAG	Agentic RAG
Retrieval	One retrieval pass	One or more passes as needed
Query handling	Query generally used as given	Query can be reformulated or decomposed
Sources	Usually one fixed knowledge base	Agent can choose among vector store, web, SQL, APIs, etc.
Stopping	Retrieval followed by generation	Agent decides when evidence is sufficient
Latency/cost	Lower	Higher
Best suited for	Simple, well-scoped factual lookup	Multi-hop, ambiguous, open-ended tasks

16. GraphRAG

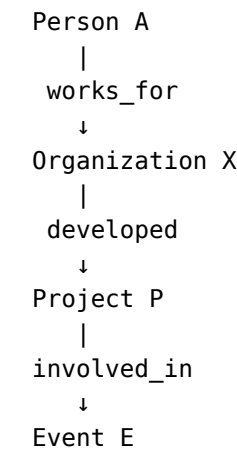
16.1 Why a graph?

Standard RAG retrieves a set of text passages that are individually similar to the query. The lecture points out that these passages arrive as a flat list and do not inherently record how the facts in the passages are related. This can become a problem for questions that span many documents and require relationships between entities to be followed.

GraphRAG addresses this by building a knowledge graph from the corpus. Entities become nodes, relationships become edges, and groups of related entities are organized into communities. The system can then retrieve a connected graph neighborhood or a summary of a community instead of treating every passage as an independent item.

16.2 Entities and relationships

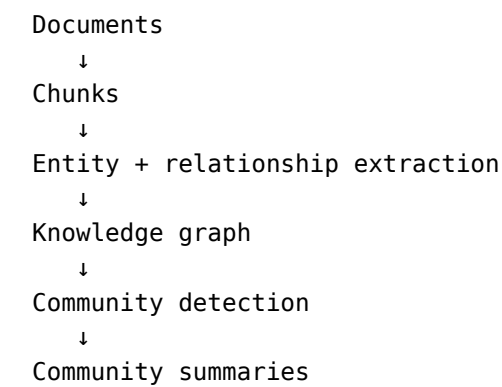
Suppose a corpus contains the statements that Person A works for Organization X, Organization X developed Project P, and Project P was involved in Event E. A text-only retrieval system may find the individual statements separately. A graph makes the relationship explicit:



This representation is particularly useful when the question itself is relational, such as asking how two entities are connected or what sequence of relationships links them.

16.3 GraphRAG indexing pipeline

The lecture describes a one-time indexing process in which an LLM reads the corpus, extracts entities and relationships, constructs a knowledge graph, identifies communities using clustering, and creates summaries of those communities. [filecite]turn0file0[L368-L389]



16.4 Community detection

A large graph can contain many tightly connected groups. The lecture specifically mentions Leiden clustering as a method for identifying communities. The communities can then be summarized hierarchically, allowing the system to support not only local entity questions but also broader questions about themes across the corpus. [filecite]turn0file0[L376-L389]

16.5 Why GraphRAG costs more during indexing

Standard vector RAG can often create embeddings once and then search the resulting vector index. GraphRAG requires substantially more processing because chunks are passed through an LLM to extract entities and relationships, and communities are passed through an LLM again for summarization. The lecture explicitly states that indexing is where GraphRAG spends much of its cost, while later queries can benefit from the already constructed graph and summaries. [filecite]turn0file0[L388-L389]

17. Standard RAG vs Agentic RAG vs GraphRAG

Property	Standard RAG	Agentic RAG	GraphRAG
Core representation	Embedded text chunks	Can use multiple retrieval/tool sources	Entities, relationships, communities
Typical retrieval	Top-k similar passages	Repeated/reformulated retrieval as needed	Graph neighborhood or community summaries
Reasoning pattern	Direct evidence lookup	Iterative goal-directed process	Relationship- and community-aware retrieval
Best use case	Pinpoint factual lookup	Multi-step or ambiguous research	Multi-hop and corpus-wide relational questions
Main cost	Retrieval + generation	Multiple LLM/tool calls	Expensive indexing and graph maintenance
Corpus update	Relatively straightforward	Depends on tools	Graph may need rebuilding/updating

The lecture's final comparison identifies standard RAG as strong for pinpoint lookups, while GraphRAG is useful for joining facts across documents and answering whole-corpus questions. It also emphasizes the higher indexing cost of GraphRAG. [filecite\turn0file0\L393-L419](#)

17.1 How to identify the right architecture in an exam question

- If the question says “retrieve the most relevant passages and answer,” think standard RAG.
- If the question says “reformulate the query, call different tools, inspect results, and continue until enough evidence is found,” think Agentic RAG.
- If the question emphasizes entities, relationships, communities, multi-hop connections, or whole-corpus themes, think GraphRAG.
- If the question emphasizes changing model behavior using very few trainable parameters, think LoRA.
- If the question gives examples in the prompt but says the model parameters remain frozen, think in-context learning.

18. High-Value Formula Sheet

18.1 Self-attention

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

$$A = QK^T$$

$$\text{Attention}(Q, K, V)$$

$$= \text{softmax}(QK^T / \sqrt{d_k}) V$$

18.2 Dimensions

$$X : T \times D$$

$$Q, K, V : T \times d$$

$$QK^T : T \times T$$

$$\text{Attention output per head} : T \times d$$

18.3 Multi-head attention

$$E_k = \text{softmax}(Q_k K_k^T / \sqrt{d_k}) V_k$$

$$E = [E_1 \ E_2 \ \dots \ E_h]$$

$$d = h \times d_k \quad (\text{typical equal-width heads})$$

18.4 Autoregressive language modeling

$$P(x_1, \dots, x_T)$$

$$= \prod_t P(x_t \mid x_1, \dots, x_{(t-1)})$$

18.5 RAG

$$P(y \mid x)$$

$$= \sum_{(d \in \mathcal{C})} P(y \mid x, d) P(d \mid x)$$

Top-k approximation:

$$P(y \mid x)$$

$$\approx \sum_{(i=1..k)} P(y \mid x, d_i) P(d_i \mid x)$$

18.6 MIPS

$$q(x) = \text{Enc}_q(x)$$

$$d(z) = \text{Enc}_d(z)$$

$$\text{score}(x, z) = q(x)^T d(z)$$

$$Z_k(x) = \text{top-}k_z \ q(x)^T d(z)$$

18.7 LoRA

$$\Delta W = BA$$

$$W = W_0 + \alpha BA$$

$$\text{Full trainable parameters} = Dd$$

$$\text{LoRA trainable parameters} = r(D+d)$$

19. NPTEL Exam Traps and Conceptual Distinctions

19.1 Attention-score matrix versus output matrix

The score matrix QK^T has dimension $T \times T$ because every query is compared with every key. The attention output has dimension $T \times d$ per head because the $T \times T$ weight matrix multiplies a $T \times d$ value matrix. Confusing these two matrices is one of the most common mistakes in dimension questions.

19.2 Similarity versus information aggregation

The query-key dot product determines the scores used to decide which positions are relevant. The values are the vectors that are actually aggregated. Thus, keys participate in matching, while values supply the information that is combined into the output.

19.3 Causal versus ordinary self-attention

Self-attention as a mechanism can allow every token to attend to every other token. Decoder-only autoregressive language modeling adds a causal restriction so that future positions cannot influence a prediction. Therefore, “self-attention” and “causal self-attention” should not be treated as synonymous.

19.4 ICL versus LoRA

Both can adapt a pretrained model to a task, but they do so differently. In-context learning keeps the parameters frozen and supplies examples through the context. LoRA keeps the original model weights frozen but introduces and trains a small low-rank parameter update. The latter is a genuine parameter-learning process; the former is not.

19.5 RAG versus LoRA

RAG and LoRA solve different problems. RAG provides external information at inference time. LoRA changes the model's parameters in a parameter-efficient manner. A system can use both, but they should not be considered interchangeable.

19.6 Standard RAG versus Agentic RAG

The defining distinction is not merely that an agent can use a vector database. The lecture's distinction is iterative decision-making: standard RAG performs a retrieval pass, whereas agentic RAG can reformulate or decompose queries, select among tools, retrieve multiple times, evaluate results, and stop when evidence is sufficient.

19.7 Standard RAG versus GraphRAG

GraphRAG is not simply a larger vector database. Its distinguishing feature is the explicit representation of entities and relationships and the use of graph communities and summaries. This makes it particularly useful when the desired answer depends on connecting facts across documents or understanding the corpus at a higher level.

20. Practice Questions with Detailed Answers

Q1. If X is $T \times D$ and WQ is $D \times d$, what is Q ?

Answer: $Q = XWQ$, so Q has dimension $T \times d$. The inner dimensions D cancel in the matrix multiplication.

NPTEL EXAM NOTE: If the answer choices contain $D \times d$, $T \times D$, and $T \times d$, select $T \times d$.

Q2. If Q and K are both $T \times d$, what is the dimension of QK^T ?

K^T is $d \times T$. Therefore QK^T is $(T \times d)(d \times T) = T \times T$. The result contains one query-key score for every pair of token positions.

Q3. Why does attention use softmax?

Softmax converts raw compatibility scores into normalized nonnegative weights. This makes it possible to form a weighted sum of value vectors. Larger scores receive larger weights, while the weights across the relevant positions sum to one.

Q4. Why is scaled dot-product attention divided by $\sqrt{d_k}$?

The scaling controls the magnitude of the dot products as the representation dimension grows. This helps prevent excessively large logits from making softmax too sharply concentrated and supports stable learning.

Q5. What does causal masking accomplish?

Causal masking prevents a token from attending to future positions. This is necessary for autoregressive next-token prediction because the model must predict a token without being given future target tokens.

Q6. What is the difference between zero-shot and few-shot ICL?

Zero-shot ICL supplies an instruction without worked examples. Few-shot ICL supplies multiple input-output examples that demonstrate the task pattern. In both cases, the model can perform the task without updating its parameters during the interaction.

Q7. A matrix W_0 is 2000×1000 and LoRA rank r is 10. How many trainable parameters are introduced?

$$\begin{aligned} & r(D+d) \\ &= 10(2000 + 1000) \\ &= 30,000 \end{aligned}$$

Answer: 30,000 trainable parameters under the lecture's LoRA parameter-count formulation, compared with 2,000,000 parameters for full updating of W_0 .

Q8. What does $P(d|x)$ represent in the RAG equation?

It represents the retrieval distribution or relevance of document d given the query x . It belongs to the retrieval side of the RAG system.

Q9. Why is top-k retrieval used?

The full corpus can be extremely large, so conditioning on every possible document is computationally impractical. Top-k retrieval approximates the full sum using only the highest-ranked documents.

Q10. What is MIPS?

MIPS stands for Maximum Inner Product Search. In the lecture's formulation, the query and document are encoded into vectors and scored using their inner product $q(x)^T d(z)$. The top-k highest-scoring documents are retrieved.

Q11. What is the defining property of Agentic RAG?

Agentic RAG can perform multiple retrieval or tool-use cycles. The agent can reformulate or decompose the query, inspect results, choose another source, and continue until it determines that sufficient evidence has been gathered.

Q12. Why can GraphRAG answer relationship-heavy questions better than flat top-k passage retrieval?

A flat list of passages does not explicitly encode the relationships among the entities mentioned in different passages. GraphRAG constructs entities as nodes and relationships as edges, allowing connected structures and communities to be retrieved. This makes multi-hop and whole-corpus questions more naturally represented.

Q13. Which method is most directly associated with low-rank matrices A and B ?

LoRA. The adapted matrix is represented as $W = W_0 + \alpha BA$, with W_0 frozen and A and B learned.

Q14. Which approach uses external knowledge without requiring that knowledge to be permanently encoded in model weights?

RAG. Retrieved documents are placed into the model's context as non-parametric memory.

Q15. Which approach is explicitly described as having higher latency because it may involve multiple LLM and retrieval calls?

Agentic RAG. The lecture contrasts it with standard RAG, which uses a lower-cost one-retrieval-plus-generation pattern.

21. Final Revision Checklist

Before attempting an NPTEL examination on this material, you should be able to explain every item below in your own words and, where applicable, write the corresponding equation without looking at the notes.

- Explain why a Transformer needs contextual representations rather than only independent token embeddings.
- Define Query, Key, and Value and explain their roles.
- Derive the dimensions of Q, K, V, and QK^T .
- Write the complete scaled dot-product attention equation.
- Explain why the scaling term is $\sqrt{d_k}$.
- Explain what softmax does to attention scores.
- Explain why the final attention output is a weighted combination of values.
- Explain the purpose of multiple attention heads.
- Compute d_k when d and the number of heads are given.
- Explain residual connections and LayerNorm.
- Explain why decoder-only LLMs use causal attention.
- Write the autoregressive factorization of sequence probability.
- Explain next-token prediction as a self-supervised objective.
- Define parametric memory and state its four limitations listed in the lecture.
- Define zero-shot, one-shot, and few-shot in-context learning.
- Explain why ICL does not require updating model weights.
- Write the LoRA equation $W = W_0 + \alpha BA$.
- Calculate full fine-tuning and LoRA parameter counts.
- Explain why LoRA is considered parameter-efficient fine-tuning.
- Draw the complete RAG pipeline from query to generated response.
- Explain the RAG probability equation and identify the roles of $P(d|x)$ and $P(y|x,d)$.
- Explain why top-k retrieval approximates full-corpus conditioning.
- Define MIPS and calculate a simple inner product.
- Distinguish RAG from ICL, LoRA, and full fine-tuning.
- Explain the Agentic AI goal-action-observation loop.
- Compare standard RAG and Agentic RAG in terms of retrieval, query handling, tools, stopping, cost, and use cases.
- Explain why GraphRAG represents entities and relationships.
- Describe the GraphRAG indexing pipeline.
- Explain community detection and community summaries.
- Compare Standard RAG, Agentic RAG, and GraphRAG.

If you can answer all of these without relying on memorized phrases, you have covered the conceptual core of the uploaded lectures. For numerical questions, repeatedly practice matrix dimensions and LoRA parameter counts. For conceptual questions, focus especially on the distinctions between parametric memory, in-context learning, LoRA, RAG, Agentic RAG, and GraphRAG.

Source and Lecture-Page Map

This guide is based primarily on the two uploaded lecture PDFs. The explanations have been expanded into full sentences and paragraphs for study purposes, while the terminology and core structure follow the lecture material.

Topic	Uploaded lecture location
Transformer building blocks and self-attention	Part I PDF, pages 3–9
Multi-head attention	Part I PDF, pages 10–13
Transformer processing layer	Part I PDF, page 14
Decoder-only models and loss	Part I PDF, pages 15–17
Large Language Models	Part I PDF, page 18
In-context learning	Part I PDF, pages 23–27
Low-rank adaptation / LoRA	Part I PDF, pages 28–31
Parametric memory and limitations	RAG PDF, pages 3–4
RAG pipeline	RAG PDF, pages 5–8
RAG mathematical formulation	RAG PDF, page 9
Retriever and MIPS	RAG PDF, pages 10–12
Agentic AI	RAG PDF, pages 13–15
GraphRAG	RAG PDF, pages 16–18

The first uploaded RAG lecture identifies the speaker as Sriram Ganapathy, Associate Professor, Electrical Engineering, IISc. [filecite]turn0file0[L2-L7] The second uploaded lecture identifies the series as “Topics in Large Language Models — Lecture Series - Part I” and attributes it to Sriram Ganapathy, Electrical Engineering, IISc. [filecite]turn0file1[L2-L10]

Important source boundary: where the lecture slides present a compact formula or diagram, this guide expands the intuition and step-by-step explanation. Those expansions are explanatory teaching material; the lecture's stated claims, terminology, equations, and comparisons remain the primary source for the study guide.